



Programa Especialización en Ciencias de Datos

Proyecto Módulo 02

Profesor: Alonso

Estudiante:

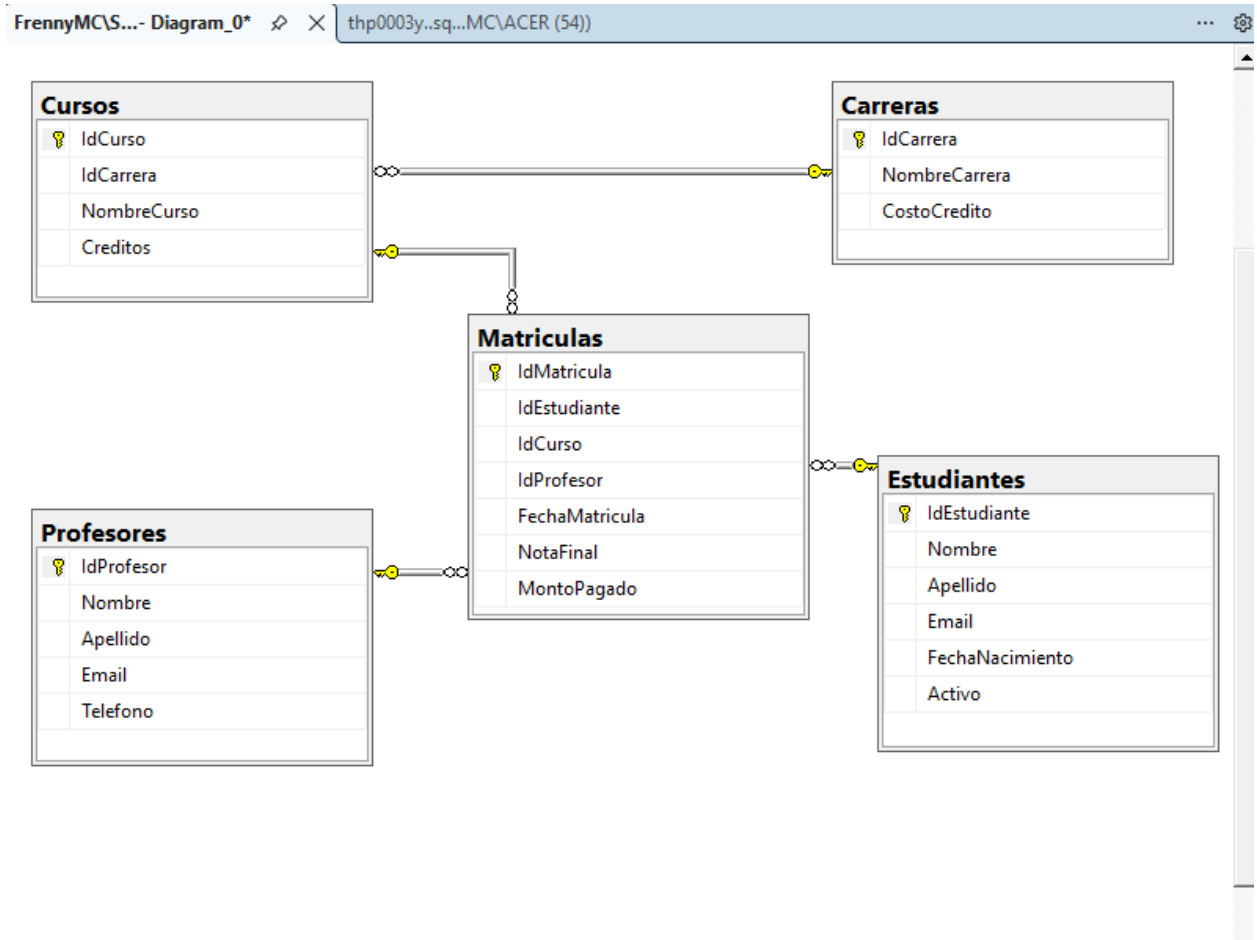
Frenny Montezuma Castillo

Fecha de entrega: 01 – 08 – 26

Tabla de Contenido

Modelo Entidad - Relación	3
Script	5
-- 1. CREACIÓN DE LA BASE DE DATOS	5
-- 2. CREACIÓN DE TABLAS	5
-- 3. INSERCIÓN DE DATOS.....	7
-- 4. VISTAS.....	8
-- 5. CONSULTAS REQUERIDAS	10
-- SUBCONSULTAS	12

Modelo Entidad - Relación



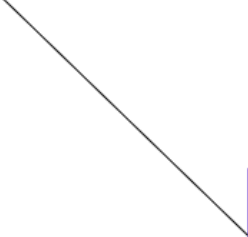
CARRERAS		
int	IdCarrera	PK
string	NombreCarrera	
decimal	CostoCredito	

ESTUDIANTES		
int	IdEstudiante	PK
string	Nombre	
string	Apellido	
string	Email	
date	FechaNacimiento	
bit	Activo	

CURSOS		
int	IdCurso	PK
int	IdCarrera	FK
string	NombreCurso	
int	Creditos	

PROFESORES		
int	IdProfesor	PK
string	Nombre	
string	Apellido	
string	Email	
string	Telefono	

MATRICULAS		
int	IdMatricula	PK
int	IdEstudiante	FK
int	IdCurso	FK
int	IdProfesor	FK
date	FechaMatricula	
decimal	NotaFinal	
decimal	MontoPagado	



Script

USE master;

GO

-- Si la base de datos ya existe, la elimina limpiamente para volver a crearla

IF EXISTS (SELECT name FROM sys.databases WHERE name = N'UniversidadDB')

BEGIN

ALTER DATABASE UniversidadDB SET SINGLE_USER WITH ROLLBACK
IMMEDIATE;

DROP DATABASE UniversidadDB;

END

GO

-- 1. CREACIÓN DE LA BASE DE DATOS

CREATE DATABASE UniversidadDB;

GO

USE UniversidadDB;

GO

-- 2. CREACIÓN DE TABLAS

-- Tabla 1: Carreras

CREATE TABLE Carreras (
 IdCarrera INT IDENTITY(1,1) PRIMARY KEY,
 NombreCarrera VARCHAR(100) NOT NULL,
 CostoCredito DECIMAL(10,2) NOT NULL
);

-- Tabla 2: Profesores

CREATE TABLE Profesores (

```
IdProfesor INT IDENTITY(1,1) PRIMARY KEY,  
Nombre VARCHAR(50) NOT NULL,  
Apellido VARCHAR(50) NOT NULL,  
Email VARCHAR(100) UNIQUE,  
Telefono VARCHAR(15) NULL  
);
```

-- Tabla 3: Estudiantes

```
CREATE TABLE Estudiantes (  
    IdEstudiante INT IDENTITY(1,1) PRIMARY KEY,  
    Nombre VARCHAR(50) NOT NULL,  
    Apellido VARCHAR(50) NOT NULL,  
    Email VARCHAR(100) UNIQUE,  
    FechaNacimiento DATE NOT NULL,  
    Activo BIT DEFAULT 1 -- 1 = Activo, 0 = Inactivo  
);
```

-- Tabla 4: Cursos

```
CREATE TABLE Cursos (  
    IdCurso INT IDENTITY(1,1) PRIMARY KEY,  
    IdCarrera INT NOT NULL,  
    NombreCurso VARCHAR(100) NOT NULL,  
    Creditos INT NOT NULL,  
    CONSTRAINT FK_Cursos_Carreras FOREIGN KEY (IdCarrera) REFERENCES  
Carreras(IdCarrera)  
);
```

-- Tabla 5: Matriculas

```
CREATE TABLE Matriculas (  
    IdMatricula INT IDENTITY(1,1) PRIMARY KEY,  
    IdEstudiante INT NOT NULL,  
    IdCurso INT NOT NULL,
```

```
IdProfesor INT NOT NULL,  
FechaMatricula DATE NOT NULL,  
NotaFinal DECIMAL(4,2) NULL,  
MontoPagado DECIMAL(10,2) NOT NULL,  
CONSTRAINT FK_Matriculas_Estudiantes FOREIGN KEY (IdEstudiante) REFERENCES  
Estudiantes(IdEstudiante),  
CONSTRAINT FK_Matriculas_Cursos FOREIGN KEY (IdCurso) REFERENCES  
Cursos(IdCurso),  
CONSTRAINT FK_Matriculas_Profesores FOREIGN KEY (IdProfesor) REFERENCES  
Profesores(IdProfesor)  
);  
GO
```

-- 3. INSERCIÓN DE DATOS

```
INSERT INTO Carreras (NombreCarrera, CostoCredito) VALUES  
('Ingeniería de Sistemas', 50.00),  
('Administración de Empresas', 40.00),  
('Derecho', 45.00);
```

```
INSERT INTO Profesores (Nombre, Apellido, Email, Telefono) VALUES  
('Carlos', 'Mendoza', 'cmendoza@universidad.edu', '8888-1111'),  
('Ana', 'Gómez', 'agomez@universidad.edu', '8888-2222'),  
('Luis', 'Alvarado', 'lalvarado@universidad.edu', NULL);
```

```
INSERT INTO Estudiantes (Nombre, Apellido, Email, FechaNacimiento, Activo) VALUES  
('Juan', 'Pérez', 'jperez@gmail.com', '2000-05-15', 1),  
('María', 'Rojas', 'mrojas@gmail.com', '1998-10-20', 1),  
('Pedro', 'Sánchez', 'psanchez@hotmail.com', '2002-01-10', 1),
```

```
('Laura', 'Castro', 'lcastro@outlook.com', '1995-12-05', 0),  
( 'Diego', 'Morales', 'dmorales@gmail.com', '2001-07-22', 1);
```

```
INSERT INTO Cursos (IdCarrera, NombreCurso, Creditos) VALUES  
(1, 'Bases de Datos I', 4),  
(1, 'Programación Web', 4),  
(2, 'Contabilidad General', 3),  
(3, 'Derecho Civil', 3);
```

```
INSERT INTO Matriculas (IdEstudiante, IdCurso, IdProfesor, FechaMatricula, NotaFinal,  
MontoPagado) VALUES  
(1, 1, 1, '2026-01-10', 85.50, 200.00),  
(1, 2, 2, '2026-01-11', 90.00, 200.00),  
(2, 1, 1, '2026-01-12', 68.00, 200.00),  
(3, 3, 3, '2026-01-15', NULL, 120.00),  
(5, 1, 1, '2026-01-20', 95.00, 200.00);  
GO
```

```
-- -----  
-- 4. VISTAS  
-- -----
```

```
-- Vista 1: Detalle completo de matrículas activas
```

```
CREATE OR ALTER VIEW vw_DetalleMatriculas AS
```

```
SELECT M.IdMatricula, E.Nombre + ' ' + E.Apellido AS Estudiante, C.NombreCurso, P.Nombre  
+ ' ' + P.Apellido AS Profesor, M.FechaMatricula, M.MontoPagado
```

```
FROM Matriculas M
```

```
INNER JOIN Estudiantes E ON M.IdEstudiante = E.IdEstudiante
```

```
INNER JOIN Cursos C ON M.IdCurso = C.IdCurso
```

```
INNER JOIN Profesores P ON M.IdProfesor = P.IdProfesor;
```

```
GO
```

-- Vista 2: Rendimiento académico (Promedio de notas por estudiante)

```
CREATE OR ALTER VIEW vw_PromedioEstudiantes AS
SELECT E.IdEstudiante, E.Nombre, E.Apellido, AVG(M.NotaFinal) AS PromedioNotas
FROM Estudiantes E
INNER JOIN Matriculas M ON E.IdEstudiante = M.IdEstudiante
WHERE M.NotaFinal IS NOT NULL
GROUP BY E.IdEstudiante, E.Nombre, E.Apellido;
GO
```

-- Vista 3: Recaudación total por cada carrera

```
CREATE OR ALTER VIEW vw_IngresosPorCarrera AS
SELECT Ca.NombreCarrera, SUM(M.MontoPagado) AS TotalRecaudado
FROM Matriculas M
INNER JOIN Cursos Cu ON M.IdCurso = Cu.IdCurso
INNER JOIN Carreras Ca ON Cu.IdCarrera = Ca.IdCarrera
GROUP BY Ca.NombreCarrera;
GO
```

-- Vista 4: Lista de estudiantes que están activos

```
CREATE OR ALTER VIEW vw_EstudiantesActivos AS
SELECT IdEstudiante, Nombre, Apellido, Email, FechaNacimiento
FROM Estudiantes
WHERE Activo = 1;
GO
```

-- Vista 5: Cantidad de alumnos matriculados por curso

```
CREATE OR ALTER VIEW vw_CursosMatriculados AS
```

```
SELECT C.NombreCurso, COUNT(M.IdMatricula) AS TotalAlumnos
FROM Cursos C
LEFT JOIN Matriculas M ON C.IdCurso = M.IdCurso
GROUP BY C.NombreCurso;
GO
```

```
-- Vista 6: Profesores con sus teléfonos de contacto asignados
CREATE OR ALTER VIEW vw_ContactoProfesores AS
SELECT IdProfesor, Nombre, Apellido, Email, Telefono
FROM Profesores
WHERE Telefono IS NOT NULL;
GO
```

```
-- 5. CONSULTAS REQUERIDAS
```

```
-- SELECT, WHERE, ORDER BY, TOP
SELECT TOP 3 Nombre, Apellido, FechaNacimiento
FROM Estudiantes
WHERE Activo = 1
ORDER BY FechaNacimiento DESC;
```

```
-- DISTINCT
SELECT DISTINCT IdCarrera FROM Cursos;
```

```
-- LIKE, AND, OR
SELECT * FROM Estudiantes
WHERE (Nombre LIKE 'J%' OR Apellido LIKE 'R%') AND Activo = 1;
```

-- BETWEEN, IN, NOT

```
SELECT * FROM Matriculas
WHERE NotaFinal BETWEEN 70 AND 100
AND IdCurso NOT IN (3, 4);
```

-- IS NULL / IS NOT NULL

```
SELECT * FROM Profesores WHERE Telefono IS NULL;
SELECT * FROM Matriculas WHERE NotaFinal IS NOT NULL;
```

-- Agregaciones: COUNT, SUM, AVG, MIN, MAX con GROUP BY y HAVING

```
SELECT IdCurso,
       COUNT(IdMatricula) AS CantidadAlumnos,
       SUM(MontoPagado) AS Recaudacion,
       AVG(NotaFinal) AS Promedio,
       MIN(NotaFinal) AS NotaMinima,
       MAX(NotaFinal) AS NotaMaxima
FROM Matriculas
GROUP BY IdCurso
HAVING COUNT(IdMatricula) > 0;
```

-- Combinaciones: INNER JOIN, LEFT JOIN, RIGHT JOIN

```
SELECT E.Nombre, C.NombreCurso
FROM Matriculas M
INNER JOIN Estudiantes E ON M.IdEstudiante = E.IdEstudiante
INNER JOIN Cursos C ON M.IdCurso = C.IdCurso;

SELECT E.Nombre, E.Apellido, M.IdMatricula
```

```
FROM Estudiantes E  
LEFT JOIN Matriculas M ON E.IdEstudiante = M.IdEstudiante;
```

```
SELECT P.Nombre, P.Apellido, M.IdMatricula  
FROM Matriculas M  
RIGHT JOIN Profesores P ON M.IdProfesor = P.IdProfesor;
```

```
-- -----
```

```
-- SUBCONSULTAS
```

```
-- -----
```

```
SELECT Nombre, Apellido  
FROM Estudiantes  
WHERE IdEstudiante IN (  
    SELECT IdEstudiante  
    FROM Matriculas  
    WHERE NotaFinal > (SELECT AVG(NotaFinal) FROM Matriculas WHERE NotaFinal IS  
NOT NULL)  
);
```